IO Streams

MUSANINYANGE M.Larisse

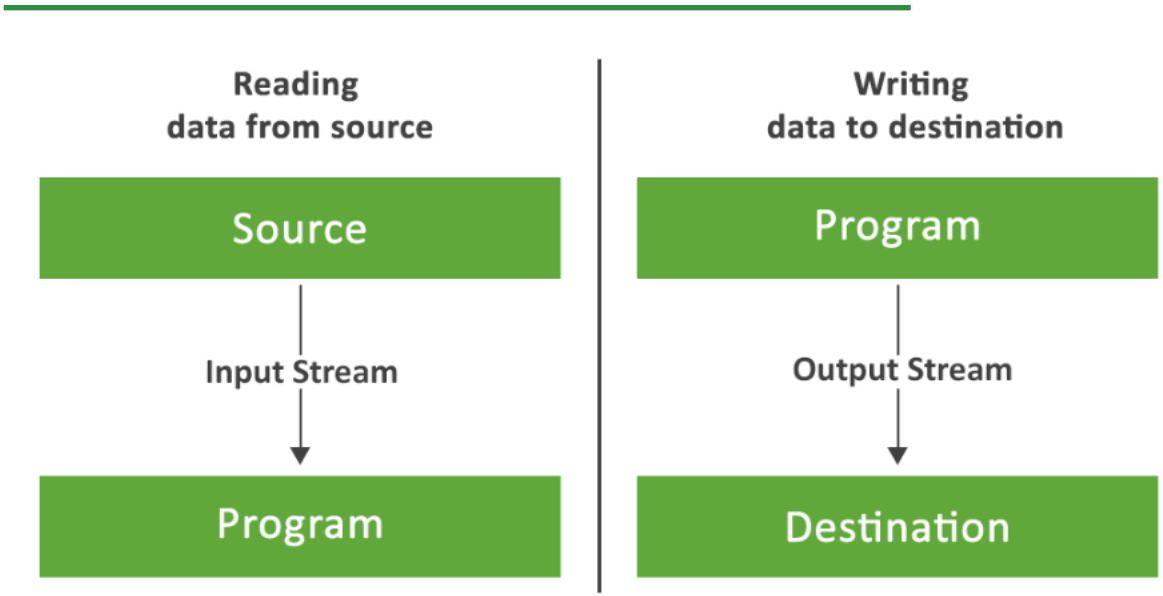
INPUT/ OUTPUT STREAMS

- Byte streams
- Character streams
- Buffered streams
- Scanning and formatting streams
- Data streams
- Object streams

What is a stream?

- In the programming world, a stream can be described as a series of data. It is known as a stream because it is similar to a stream of water that continues to flow.
- Java IO streams are flows of data that a user can either read from or write to.
- Unlike arrays, Java I/O streams do not have indexing to read or write data.
- A stream is attached to a data origin or a data target.

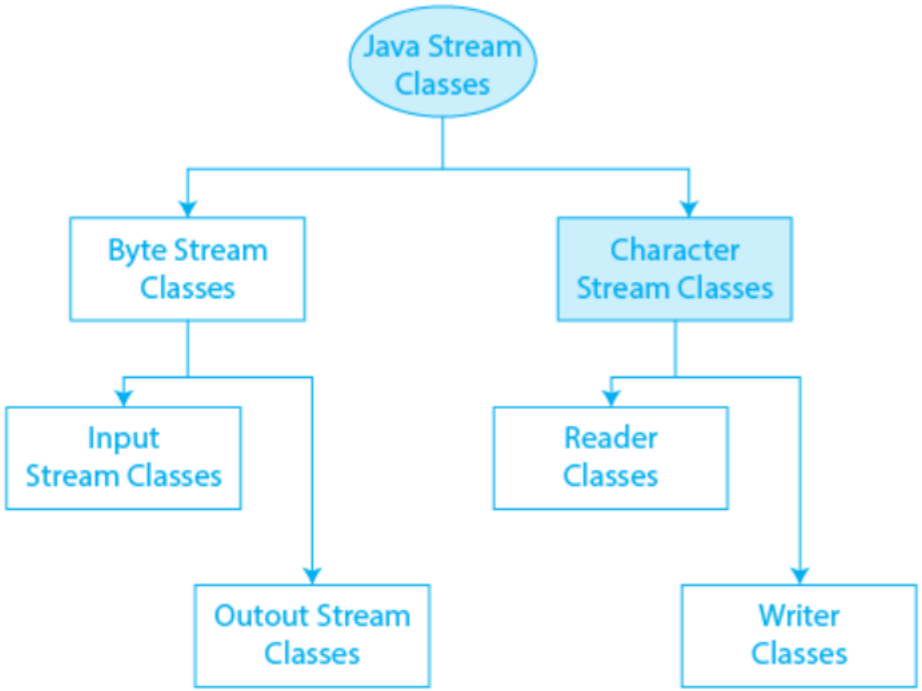
Stream


Reading
data from source
Source
Input Stream
Program
Writing
data to destination
Program
Output Stream
Destination

Types of streams

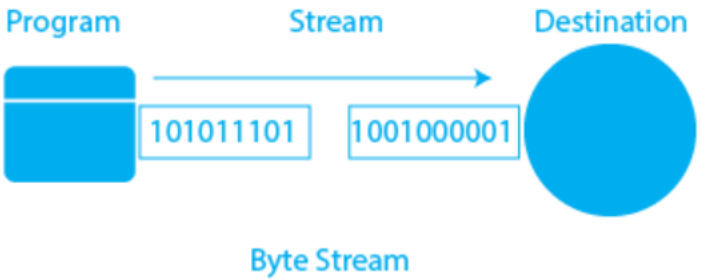
Depending upon the data a stream holds, it can be classified into:

- Byte Stream
- Character Stream



Byte Streams

- **Byte Stream classes** are used to read **bytes** from an **input stream** and write **bytes** to an **output stream**.
- To put it another way, **Byte Stream classes** read and write **8-bit binary data**.
  They are used for handling **non-text data** such as **audio, video, and image files**.



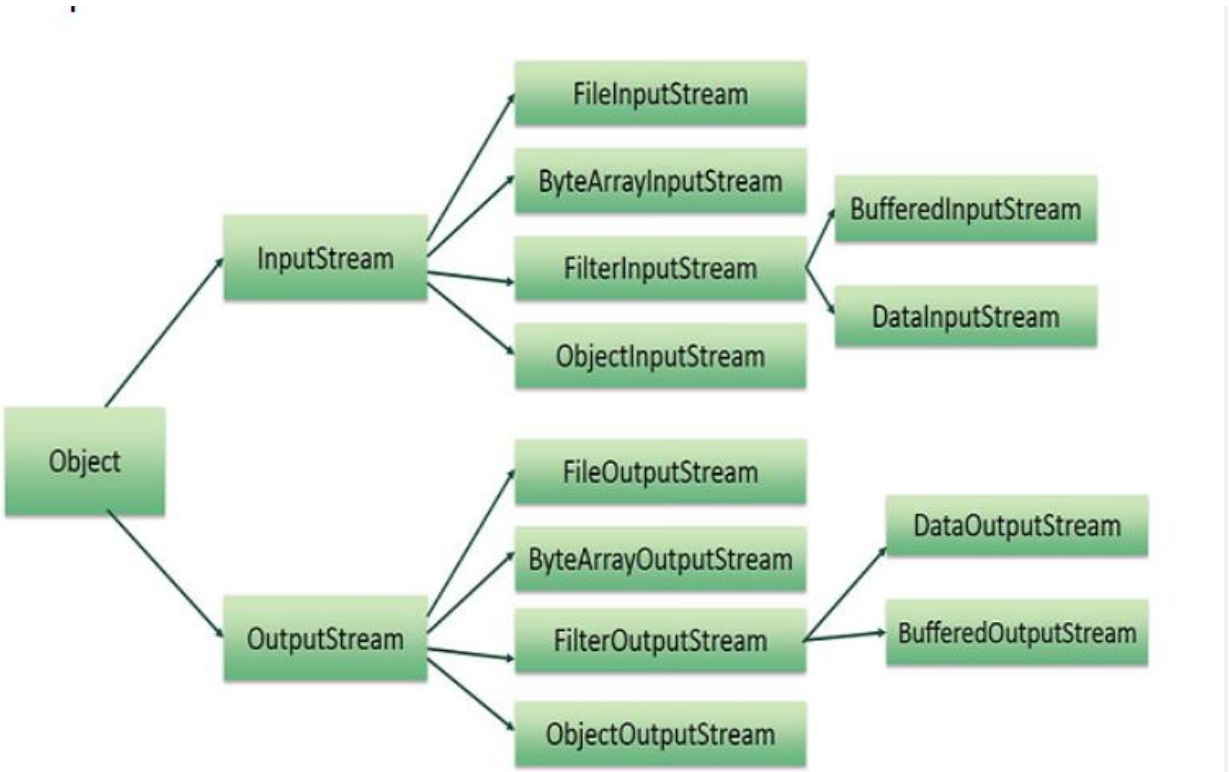
Byte streams

The ByteStream classes are classified into two types: InputStream and OutputStream. These are the abstract super classes for all Input/Output stream classes.

1. InputStream:

- This type of java stream reads data from a source. An InputStream can read data from a file, network connection, or any other I/O device. An InputStream can be used to read data from a file
- The outputstream is a java stream that takes data from a Java program and sends or writes it to the destination (data sink). A data flow out of a program is represented by an output. The output stream connects a Java program to a data sink

Byte streams classes



Byte streams

- The two popular streams are **FileInputStream** and **FileOutputStream**

- **FileInputStream:** This stream is used for reading data from the files.

- **Syntax :**

```
FileInputStream sourceStream = new FileInputStream("path_to_file");
```

Byte streams

2. OutputStream:

- The data flow from a data source to a Java program via an input stream. The data flow from a Java program to a data sink via an output stream

```
FileOutputStream targetStream = new FileOutputStream("path_to_file")
```

Byte streams example

```java
import java.io.*;
public class InputOutputStreams {

    public static void main(String[] args)throws IOException {
        // TODO Auto-generated method stub
     FileInputStream source=null;
     FileOutputStream target=null;
     try {
        source=new FileInputStream("C:\\external\\daily\\RCA COURSES\\JAVA\\source.txt") ;
        target=new FileOutputStream("C:\\external\\daily\\RCA COURSES\\JAVA\\destination.txt");
        int content;
        //reads a byte at time, when it reaches the of file, it returns -1
        while((content=source.read())!=-1) {
            //read() method reads a single byte from an input source
            //and returns its value as an int in the range 0 to 255 ( 0x00 - 0xff )
            target.write((byte)content);
        }
     }finally {
         source.close();
         target.close();
     }
    }

}
```

Character streams

- The Character stream is used for 16-bit Unicode input and output operations.
- Character streams would be advantageous if we wanted to copy a text file containing characters from one source to another using streams because it deals with characters. Java characters are 2 bytes (or 16 bits) in size.
- CharacterStream classes are provided by the java.io package to overcome the limitations of ByteStream classes, which can only handle 8-bit bytes and are incompatible with working directly with Unicode characters.
- CharacterStream classes are used to work with Unicode characters that are 16 bits in length. They can work with characters, char arrays, and Strings.

Character Streams

- CharacterStream classes are divided into two types for this purpose: Reader classes and Writer classes.
- There are numerous character stream classes in Java, but the most common ones are: **FileReader and FileWriter**
- **FileReader:** It is used to read two bytes at a time from the source.
- **FileWriter**
  It is used to write two bytes at a time to the destination.

Character streams example

```
import java.io.*;
public class InputOutputStreams {

    public static void main(String[] args)throws IOException {
        // TODO Auto-generated method stub
        FileReader source=null;
     FileWriter target=null;

     try {
        source=new FileReader("C:\\external\\daily\\RCA COURSES\\JAVA\\source.txt") ;
        target=new FileWriter("C:\\external\\daily\\RCA COURSES\\JAVA\\destination.txt");
        int content;
        //reading source file and writing to target file character by character
        while((content=source.read())!=-1) {

            target.write((char)content);
        }
     }finally {
         source.close();
         target.close();
     }
    }

}
```

Byte streams VS character streams

Byte Stream	Character Stream
It operates on raw binary data and data is processed byte by byte	It operates on text data and is processed character by character
Suitable for processing non-textual data such as images, videos, etc.	Suitable for processing textual data such as documents, HTML, etc.
Input and Output operations are performed using InputStream and OutputStream classes	Input and Output operations are performed using Reader and Writer classes
Suitable for low-level input and output operations	Suitable for high-level input and output operations

Reader

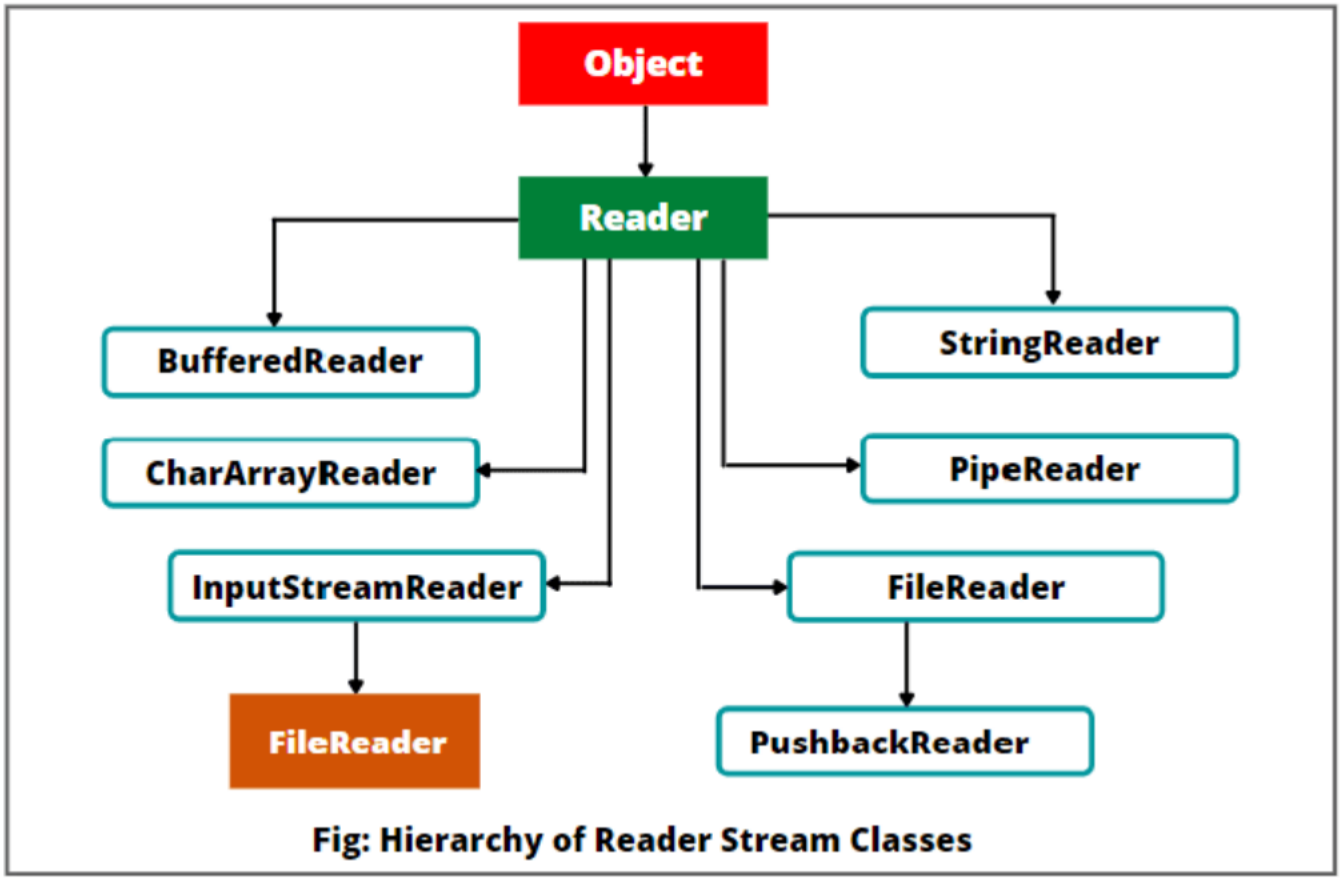


Fig: Hierarchy of Reader Stream Classes

Writer

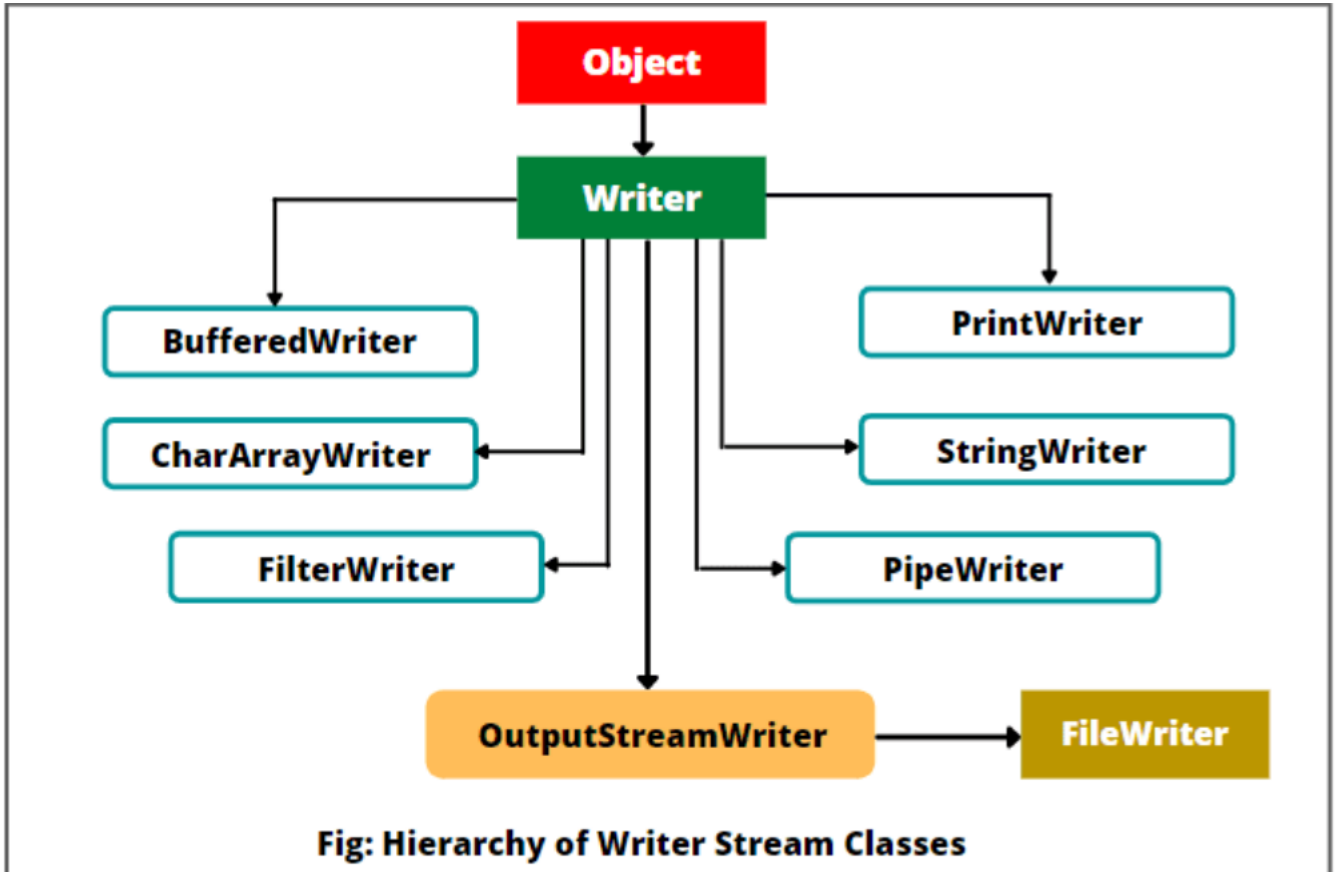


Fig: Hierarchy of Writer Stream Classes

Buffered Streams

- Buffered input streams read data from a memory area known as buffer. We send request to OS only when the buffer is empty.
- Similarly, buffered output streams write data to a buffer, and request are send to OS only when the buffer is full and there is no space further to perform write operation.
- We convert the unbuffered stream into buffered stream by wrapping to a buffered stream

```
inputStream = new BufferedReader(new FileReader("fileOne.txt"));
outputStream = new BufferedWriter(new FileWriter("fileTwo.txt"));
```

Buffered Streams

- There are four buffered stream classes used to wrap unbuffered streams: `BufferedInputStream` and `BufferedOutputStrem` create buffered byte streams , while `BufferedReader` and `BufferedWriter` create buffered character streams.

Example of Buffered streams

```
import java.io.*;
public class StreamMain {

    public static void main(String[] args)  {
        // TODO Auto-generated method stub



    try (
            BufferedInputStream bufferIn=new BufferedInputStream( new FileInputStream("C:\\external\\d
            BufferedOutputStream bufferOut=new BufferedOutputStream(new FileOutputStream("C:\\external
    {


      int content;
      //reading source file and writing to target file character by character
      while((content=bufferIn.read())!=-1) {

          bufferOut.write((byte)content);
      }

    }catch(IOException e) {
        System.out.println(e);
    }

    }

}
```

Line-Oriented I/O

- **What is a Line?**
- A **line** is a sequence of characters ending with a **line terminator**.
- Common line terminators:
  - **\r\n** → Carriage return + line feed (Windows)
  - **\r** → Carriage return only (Older Mac systems)
  - **\n** → Line feed only (Linux/Unix)

Java Classes for Line-Oriented I/O

Task	Class	Method	Notes
Read lines from file	`BufferedReader`	`readLine()`	Returns `null` at the end of file
Write lines to file	`PrintWriter`	`println()`	Adds a newline automatically
Read from console	`Scanner`	`nextLine()`	Blocks until user presses Enter

Key Points

- **BufferedReader** → Efficient line reading from files.
- **PrintWriter** → Convenient for writing full lines and formatted text.
- **Scanner** → Useful for reading lines from **console input**.

Line-Oriented I/O

```
import java.io.*;
public class StreamMain {

    public static void main(String[] args)  {
        // TODO Auto-generated method stub



    try (
            BufferedReader bufferIn=new BufferedReader( new FileReader("C:\\external\\daily\\RCA COUR
            PrintWriter out=new PrintWriter(new FileWriter("C:\\external\\daily\\RCA COURSES\\JAVA\\d
    {


      String content;
      //readline returns null when it reaches the end of the file
      while((content=bufferIn.readLine())!=null) {

        out.println(content);
      }

    }catch(IOException e) {
        System.out.println(e);
    }

    }

}
```

Scanning and Formatting

Programming I/O often involves translating to and from the neatly formatted data humans like to work with. To assist you with these chores, the Java platform provides two APIs:

- The scanner API breaks input into individual tokens associated with bits of data.
- The formatting API assembles data into nicely formatted, human-readable form.

Scanning

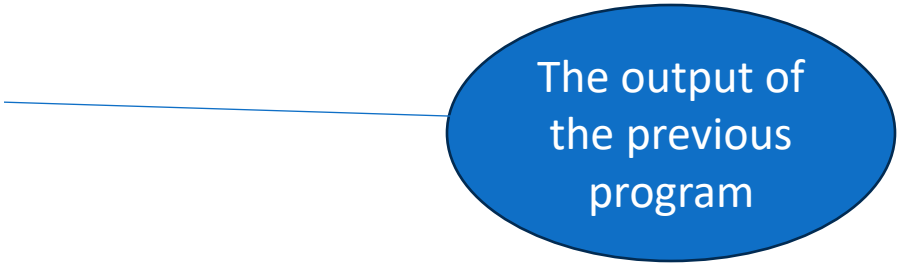
- Objects of type Scanner are useful for breaking down formatted input into tokens and translating individual tokens according to their data type.
- By default, a scanner uses white space to separate tokens((White space characters include blanks, tabs, and line terminators)
- Let's see how scanning works with an example

Scanning with example

```
import java.util.Scanner;

public class ScanningMain {

    public static void main(String[] args) {
        // TODO Auto-generated method stub


        try (
        Scanner c=new Scanner( new FileReader("C:\\external\\daily\\RCA COURSES\\JAVA\\source.txt"));
        )
        {


          while(c.hasNext()) {

            System.out.println(c.next());
          }

        }catch(IOException e) {
            System.out.println(e);
        }



    }

}
```

Scanning

```
hello
,
we
are
reading
files
in
java.
```



- NB: Even though a scanner is not a stream, you need to close it to indicate that you're done with its underlying stream. We used try with resources which closes the resources automatically

Using Delimiter

- **1. What is a Delimiter?**
- A **delimiter** is a **character or sequence of characters** that separates data items in input.
  When reading input using a **Scanner**, Java uses delimiters to **determine where one token (word/number)** ends and the next one begins.
- Think of a delimiter as a *splitter* that tells the Scanner where to stop reading.
- By default, the Scanner class uses **whitespace** (spaces, tabs, newlines) as the delimiter.

Using Delimiter

```
class Delimiter{
    public static void main(String []a) {
        String data="Irisa 18 0789374844";
        Scanner c=new Scanner(data);
        String name=c.next();
        int age=c.nextInt();
        String phone=c.next();
        System.out.println("name :"+name+" age: "+age+" phone :"+phone);

    }

}
```

Using Delimiter

```
 public static void main(String[] args) {
     // TODO Auto-generated method stub

String date="10/11/2023";
int sum=0;
     try (
     Scanner c=new Scanner( date);
     )
     {


       while(c.hasNext()) {
           c.useDelimiter("/");
           sum=c.nextInt()+sum;


       }
       System.out.println(sum);
     }
```

Scanner useLocale() method

- **What is a Locale?**
- A **Locale** in Java represents a **specific geographical, political, or cultural region**.
  It affects how numbers, dates, and text are interpreted or formatted especially symbols like **decimal separators (dot vs comma)** and **thousand separators**.

For example:

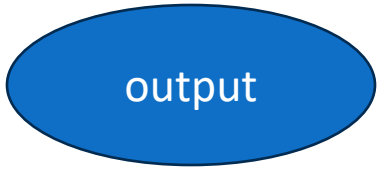
Country	Decimal Separator	Example
USA (Locale.US)	`.` (dot)	`3.14`
France (Locale.FRANCE)	`,` (comma)	`3,14`

Scanner useLocale() method

- **Why useLocale() in Scanner?**
- When reading **numbers** using a Scanner, Java expects them to follow the **default locale** of your system.
- If the input uses a **different decimal format** (e.g., a comma instead of a dot), Scanner may fail to read it correctly unless you **change the locale** using useLocale().
- Syntax : scanner.useLocale(Locale locale)

Scanner useLocale() method

```
class Locale{
    public static void main(String []a) {
        String data="3,14";
        Scanner c=new Scanner(data);
        c.useLocale(java.util.Locale.FRANCE);
        double pie=c.nextDouble();
        System.out.println("pie is:"+pie);
    }

}
```




```
pie is:3.14
```

Some primitive data types

- hasNextBoolean(): boolean -Scanner
- hasNextByte(): boolean - Scanner
- hasNext Double(): boolean - Scanner hasNextFloat(): boolean -Scanner hasNextInt()': boolean-Scanner hasNextLine(): boolean - Scanner
- hasNextLong(): boolean - Scanner

Formatting

- A PrintStream adds functionality to another output stream, namely the ability to print representations of various data values conveniently
- Unlike other output streams, a PrintStream never throws an IOException; instead, exceptional situations merely set an internal flag that can be tested via the checkError method.

Printf() method

- The .printf() method prints output to the console with the use of various formatting commands.

Syntax

```
System.out.printf(formatstring, arg1, arg2, ... argN);
```

- We specify the formatting rules using the format parameter. Rules start with the % character.

- Eg: `System.out.printf("Hello %s!%n", "World");`
- `Hello World!  ; the output produced by the above codes`

Format Rules

```
%[flags][width][.precision]conversion-character
```

- Specifiers in the brackets are optional.
- Internally, *printf()* uses the java.util.Formatter class to parse the format string and generate the output
- ***conversion-character*** **:**is required and determines how the argument is formatted.
- **The [flags]** define standard ways to modify the output and are most common for formatting integers and floating-point numbers.
- **The [width]** specifies the field width for outputting the argument. It represents the minimum number of characters written to the output.

Format Rules

- Specifiers in the brackets are optional.
- **The [.precision]** specifies the number of digits of precision when outputting floating-point values. Additionally, we can use it to define the length of a substring to extract from a String

Formatting example

```java
public class FormatingMain {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

          System.out.printf("%S","hello");
          System.out.printf("%n");
          System.out.printf("%C%n",'a');
          System.out.printf("%.2f%n",4.5);
          System.out.printf("%b%n",4<5);
          System.out.printf("%-8d",24);
          System.out.printf("%8d",24);
```

Left padding

Right padding

```
HELLO
A
4.50
true
24              24
```

output

Conversion type

conversion-characters are one of the following:

- b: Boolean value.
- B: Boolean value, uppercase.
- c: Unicode character.
- C: Unicode character, force uppercase.
- d: Decimal integer.
- f: Floating-point number.
- h: Hashcode.
- n: Newline character, platform specific.
- s: String.
- S: String, force uppercase.
- t: Time/date formatting.

Formatting a table

```
public class FormatingMain {

    public static void main(String[] args) {
        // TODO Auto-generated method stub


            System.out.printf("_____%n");
            System.out.printf("|%-10S|%-5s|%-3s|%n","names","gender","age");
            System.out.printf("_____%n");
            System.out.printf("|%-10S|%-5s|%-3d|%n","Angel","Female",17);

    }

}
```

output

```
_____
|NAMES     |gender|age|
_____
|ANGEL     |Female|17 |
```

Data Streams

- DataStreams are used to read or write primitive data types such as int, float, double, long, boolean, etc. Data streams read and write data in its binary format, which means that they are not human-readable.

- To use DataStreams, you first create an instance of the DataInputStream or DataOutputStream class, and then use its read and write methods to read and write primitive data types

Data Streams example

```
import java.io.*;

public class DataStreamManin {

    public static void main(String[] args) throws IOException{
        // TODO Auto-generated method stub

        try ( DataOutputStream dout =
                new DataOutputStream(new FileOutputStream("C:\\external\\daily\\RCA COURSES\\JAVA\\file.txt"));
                DataInputStream din =
                        new DataInputStream(new FileInputStream("C:\\external\\daily\\RCA COURSES\\JAVA\\file.txt")) ) {

    dout.writeDouble(1.1);
    dout.writeInt(55);
    dout.writeBoolean(true);
    dout.writeChar('4');
    dout.writeUTF("bonjour");

    // Illustrating readDouble() method
    double a = din.readDouble();

    // Illustrating readInt() method
    int b = din.readInt();

    // Illustrating readBoolean() method
    boolean c = din.readBoolean();

    // Illustrating readChar() method
    char d = din.readChar();
    String k=din.readUTF();
    // Print the values
    System.out.println("Values: " + a + " " + b + " " + c + " " + d+" "+k);

}
    }

}
```

Object Stream

- ObjectStreams are used to read and write objects to and from files or other streams.
- ObjectStreams read and write objects in a serialized format, which means that the objects are converted to a stream of bytes that can be written to a file or sent over a network.
- To use ObjectStreams, you first create an instance of the ObjectOutputStream or ObjectInputStream class, and then use its writeObject and readObject methods to write and read objects

Serialization

- Serialization in Java allows us to convert an Object to stream that we can send over the network or save it as file or store in DB for later usage.
- Deserialization is the reverse process where the byte stream is used to recreate the actual Java object in memory
- The byte stream created is platform independent. So, the object serialized on one platform can be deserialized on a different platform.
- To make a Java object serializable we implement the **java.io.Serializable interface**.
- The **ObjectOutputStream** class contains **writeObject()** method for serializing an Object.
- The **ObjectInputStream** class contains **readObject()** method for deserializing an object

Advantages of Serialization

1.To save/persist state of an object.

2.To travel an object across a network.

Serializable interface

- Only the objects of those classes can be serialized which are implementing **java.io.Serializable** interface. Serializable is a **marker interface** (has no data member and method). It is used to “mark” java classes so that objects of these classes may get certain capability.

SerialVersionUID

- **SerialVersionUID**: The Serialization runtime associates a version number with each Serializable class called a SerialVersionUID, which is used during Deserialization to verify that sender and receiver of a serialized object have loaded classes for that object which are compatible with respect to serialization.

- If the receiver has loaded a class for the object that has different UID than that of corresponding sender's class, the Deserialization will result in an **InvalidClassException**

Object Stream example

```
import java.io.Serializable;

public class Student implements Serializable {
 /**
      *
      */
     private static final long serialVersionUID = 1L;
String firstName;
 String lastName;
 int age;
public String getFirstName() {
     return firstName;
}
public void setFirstName(String firstName) {
     this.firstName = firstName;
}
public String getLastName() {
     return lastName;
}
public void setLastName(String lastName) {
     this.lastName = lastName;
}
public int getAge() {
     return age;
}
public void setAge(int age) {
     this.age = age;
}
public Student(String firstName, String lastName, int age) {
     super();
     this.firstName = firstName;
     this.lastName = lastName;
     this.age = age;
}
@Override
public String toString() {
     return "Student details :firstName=" + firstName + ", lastName:" + lastName + ", age:" + age ;
}

}
```

Object Stream example

```
1  import java.io.*;
2
3  public class StudentObjectStream {
4
5      public static void main(String[] args) throws IOException,ClassNotFoundException {
6          // TODO Auto-generated method stub
7
8          Student s=new Student("Lars", "Munezero", 16);
9
10         try(FileOutputStream fos = new FileOutputStream("object.bin");
11
12           ObjectOutputStream oos = new ObjectOutputStream(fos);
13                 FileInputStream fis = new FileInputStream("object.bin");
14
15                   ObjectInputStream ois = new ObjectInputStream(fis);
16                 )
17
18         {
19             oos.writeObject(s);
20
21             Student s1 = (Student)ois.readObject();
22
23               System.out.println(s1);
24
25
26         }
27     }
28
29
30
31 }
32
```

Object Stream

- In the above example, we have to catch Java ClassNotFoundException

- Java ClassNotFoundException occurs when the application tries to load a class but Classloader is not able to find it in the classpath.
- ClassNotFoundException is a checked exception, so it has to be catch or thrown to the caller

File IO

- **What is a path?**
- As name suggests Path is the particular location of an entity such as file or a directory in a file system so that one can search and access it at that particular location
- Technically in terms of Java, Path is an interface which is introduced in Java NIO file package during Java version 7,and is the representation of location in particular file system. As path interface is in Java NIO package so it get its qualified name as java.nio.file.Path.
- In general path of an entity could be of two types one is absolute path and other is relative path. As name of both paths suggests that absolute path is the location address from the root to the entity where it locates while relative path is the location address which is relative to some other path. Path uses delimiters in its definition as "\" for Windows and "/" for Unix operating systems.